



shaoyuanjie@hust.edu.cn

课程说明

- 主要教材
 - 《程序设计教程:用C/C++语言编程》
- 参考教材
 - 谭浩强, 《C程序设计》
 - 苏小红等, 《C语言程序设计》
- 理论学时 **36**
- 实验学时 **20**
 - 扩展至28学时
- 最终成绩
 - 平时成绩
 - 出勤
 - 作业
 - 实验
 - 考试成绩



课件下载



群名称: Hust-EIC-C-2023

群 号: 866760713

- PPT (理论课, 实验课)
- 习题集 (用于自我提升和复习备考)
- 作业题
- 代码 (含实验课代码)
- 教学内容讨论请私聊
- 请不要在群里聊天

- Questions?



机器语言

计算机能直接识别和接受的二进制代码称为**机器指令**。机器指令的集合就是该计算机的**机器语言**。

特点：难学，难记，难检查，难修改，难以推广使用。依赖具体机器难以移植。

```
B8 7F 01
BB 21 02
03 D8
B8 1F 04
2B C3
```

汇编语言

机器语言的符号化。用英文字母和数字表示指令的**符号语言**。

特点：相比机器语言简单好记，但仍然难以普及。汇编指令需通过**汇编程序**转换为机器指令才能被计算机执行。依赖具体机器难以移植。

```
MOV AX 383
MOV BX 545
ADD BX AX
MOV AX 1055
SUB AX BX
```

高级语言

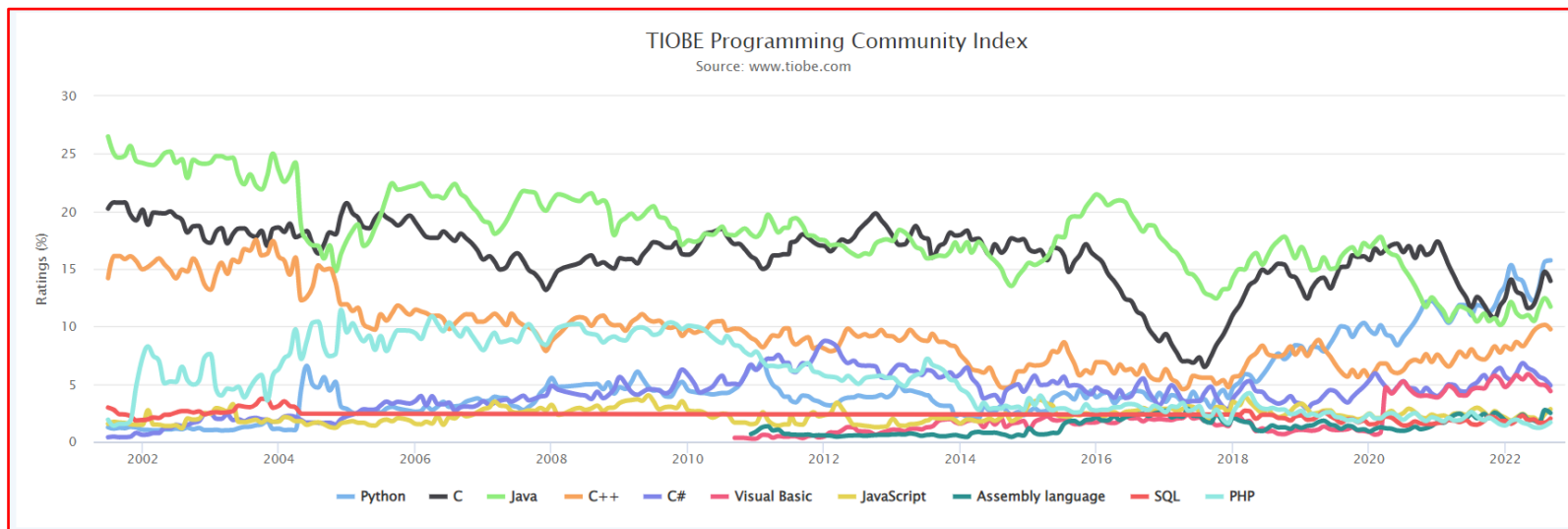
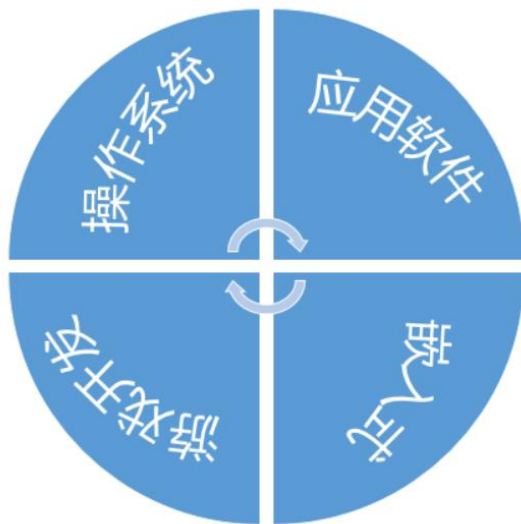
高级语言更接近于人们习惯使用的自然语言和数学语言。

特点：功能强大，不依赖于具体机器。用高级语言编写的**源程序**需要通过**编译程序**转换为机器指令的**目标程序**。

```
S=1055-(383+545)
```

0

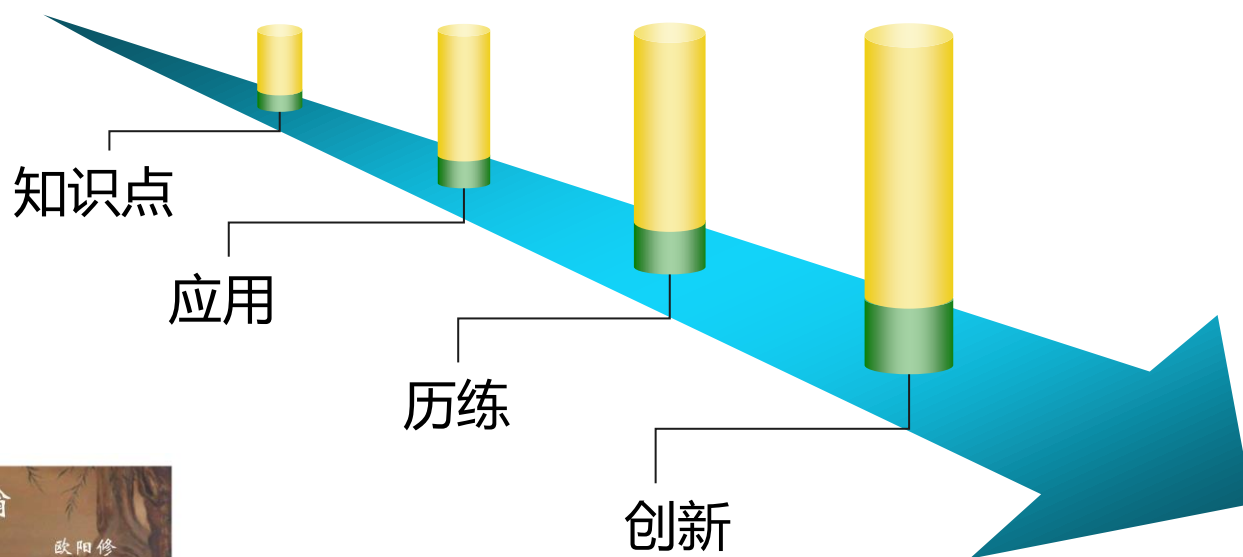
选择C语言?



选择C语言?

Sep 2022	Sep 2021	Change	Programming Language	
1	2	▲		Python
2	1	▼		C
3	3			Java
4	4			C++
5	5			C#
6	6			Visual Basic
7	7			JavaScript
8	8			Assembly language
9	10	▲		SQL
10	9	▼		PHP

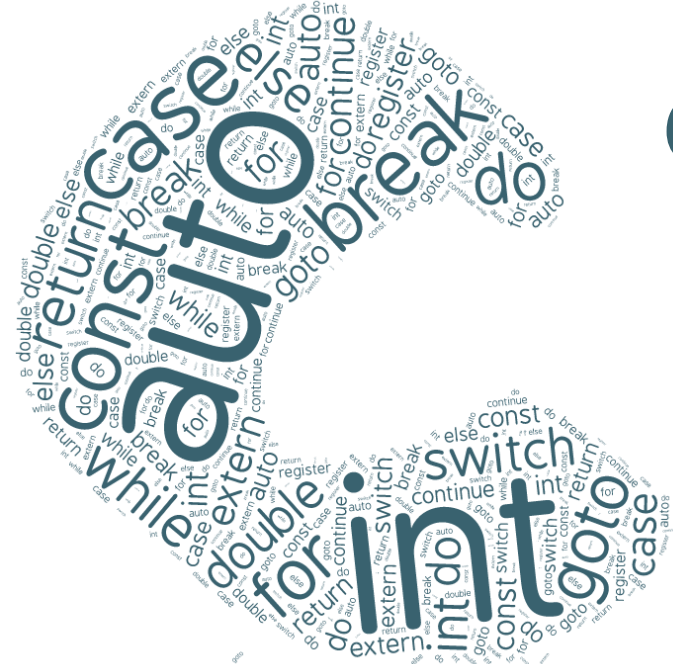
熟能生巧



“科技立则民族立，科技强则国家强”



**要时不我待
推进科技自立自强，
只争朝夕
突破“卡脖子”问题，
努力把关键核心技术和装备制造业
掌握在我们自己手里。**



chapter 1

概论

1

C语言的发展及特点

2

C语言程序的书写格式和结构特点

3

C语言的基本语法单位

4

简单的输入与输出

5

运行C程序的一般步骤



D.M.Ritchie
丹尼斯·里奇

- 1972—1973年，**美国贝尔实验室的D.M.Ritchie** 在B语言的基础上设计出了**C语言**。
- 最初C语言只是为**描述和实现UNIX操作系统**提供一种工作语言而设计的。
- 1978年以后，C语言先后移植到大、中、小和微型计算机上。C语言便很快风靡全世界，成为世界上应用最广泛的程序设计高级语言。
- 1978年，Brian W.Kernighan和Dennis M.Ritchie 合著了《The C Programming Language》，这本书中介绍的C语言成为后来广泛使用的C语言版本的基础。
- 1989年，ANSI公布了一个完整的C语言标准——ANSI X3.159—1989(常称为**ANSI C或C 89**)。
- 1999年，ISO又对C语言标准进行了修订，在基本保留原来的C语言特征的基础上，针对应用的需要，增加了一些功能，尤其是C++中的一些功能，并在2001年和2004年先后进行了两次技术修正，它被称为**C 99**，C 99是C 89的扩充。
- 2011年C语言标准委员会重新修订C语言，称为**C 11**。

1.1

C语言的特点



最简单的c语言程序

```
#include <stdio.h>           //这是编译预处理指令
int main()                   //定义主函数
{                             //函数开始的标志
    printf("This is a C program.\n"); //输出所指定的一行信息
    return 0;                //函数执行完毕时返回函数值0
}                             //函数结束的标志
```



程序分析

- **main**是函数的名字，表示“主函数”；每一个C语言程序都必须有一个 main 函数。
- main前面的**int**表示此函数的类型是int类型(整型)，即在执行主函数后会得到一个值(即函数值)，其值为整型。
- **return 0;**的作用是当main函数执行结束前将整数0作为函数值，返回到调用函数处。
- 函数体由花括号**{}**括起来。

最简单的c语言程序

```
#include <stdio.h>           //这是编译预处理指令
int main()                   //定义主函数
{                             //函数开始的标志
    printf("This is a C program.\n"); //输出所指定的一行信息
    return 0;                //函数执行完毕时返回函数值0
}                             //函数结束的标志
```



程序分析

- **printf**是C编译系统提供的函数库中的输出函数。printf函数中**双引号**内的字符串"This is a C program."按原样输出。**\n**是换行符，即在输出"This is a C program."后，显示屏上的光标位置移到下一行的开头。
- 每个语句最后都有一个**分号**，表示语句结束。

最简单的c语言程序

```
#include <stdio.h>           //这是编译预处理指令
int main()                   //定义主函数
{                             //函数开始的标志
    printf("This is a C program.\n"); //输出所指定的一行信息
    return 0;                 //函数执行完毕时返回函数值0
}                             //函数结束的标志
```



程序分析

- 在使用函数库中的输入输出函数时，编译系统要求程序提供有关此函数的信息，程序第1行“**#include <stdio.h>**”的作用就是用来提供这些信息的。**stdio.h**是系统提供的一个文件名，**stdio**是standard input & output的缩写，文件后缀**.h**的意思是头文件(header file)，因为这些文件都是放在程序各文件模块的开头的。输入输出函数的相关信息已事先放在stdio.h文件中。

最简单的c语言程序

```
#include <stdio.h>           //这是编译预处理指令
int main()                   //定义主函数
{                             //函数开始的标志
    printf("This is a C program.\n"); //输出所指定的一行信息
    return 0;                 //函数执行完毕时返回函数值0
}                             //函数结束的标志
```



程序分析

- **//**表示从此处到本行结束是“注释”，用来对程序有关部分进行必要的说明。在写C程序时应当多用注释，以方便自己和别人理解程序各部分的作用。在程序进行预编译处理时将每个注释替换为一个空格，因此在编译时注释部分不产生目标代码，注释对运行不起作用。注释只是给人看的，而不是让计算机执行的。

以//开始的单行注释

这种注释可以单独占一行，也可以出现在一行中其他内容的右侧。此种注释的范围从//开始，以换行符结束。如果注释内容一行内写不下，可以用多个单行注释。

```
//第一行注释  
//继续注释
```

```
/*一整块都是  
注释*/
```

以/*开始，以*/结束的块式注释

这种注释可以包含多行内容。它可以单独占一行(在行开头以/*开始，行末以*/结束)，也可以包含多行。编译系统在发现一个/*后，会开始找注释结束符*/，把二者间的内容作为注释。

注意：在字符串中的//和/*都不作为注释的开始。而是作为字符串的一部分。

1.2

C语言程序的书写格式和结构特点

一个简单的C语言程序

```
1  #include <stdio.h>
2
3  void main()
4  {
5      int r, h;
6      float v;
7      scanf("%d%d", &r, &h);
8      v = 3.14159 * r * r * h;
9      printf("v = %f \n", v);
10 }
11
```

```
1 2
v = 6.283180
```

- 符合C标准的程序特点（根据前面的简单程序，至少包括以下几点）

- 1) C语言程序习惯上使用小写英文字母。
- 2) C语言程序也是由一个个的语句组成。
- 3) C语言程序**不使用行序号**。
- 4) C语言程序使用分号；作为语句的终止符或分隔符。
- 5) 一般情况下，每个语句占用一个书写行的位置。
- 6) C语言程序中用大括弧对{ }，表示程序的起止或结构层次范围。
- 7) C语言程序中，为了增强可读性，可以使用适量的空格和空行。

C编译系统通常无视这样空格和空行。

1.2

C语言程序的结构特点

再看一个稍微复杂点的程序：编一程序将从键盘上输入的一串字符中的小写字母变成大写输出

```
1  /* print string as uppercase */
2  #include <stdio.h>
3  #define SIZE 80
4  void putupper(char ch);
5
6  void main()
7  {
8      char str[SIZE];
9      int i;
10     scanf("%s", str);
11     for (i = 0; str[i] != '\0'; i++)
12     {
13         putupper(str[i]);
14     }
15 }
```

```
17 void putupper(char ch)
18 {
19     char cc;
20     cc = (ch >= 'a' && ch <= 'z') ? \
21         ch + 'A' - 'a':ch;
22
23     putchar(cc);
24 }
```

abc
ABC

- C语言程序的执行是从主函数main()开始，主函数中的所有语句执行完毕，则程序执行结束。
- main()函数之外的其它函数都是在执行main()函数时，通过嵌套调用而执行的，在程序中除了可以调用用户自己编制的函数外，还可以调用由系统提供的标准函数。

(1) C程序的组成： 一个C程序可以由若干个函数构成，其中必须有且只能有一个以main命名的主函数。

(2) C函数的组成： C函数是一个独立的程序块，定义时不能嵌套。 一个函数由两个部分组成：函数头和函数体。

(3) C函数分类

两类： 标准函数，用户定义函数

用户定义函数是由程序员在自己的源程序中编写的函数。

标准函数是由C编译程序提供的一些通用函数，C标准函数又称为C库函数。

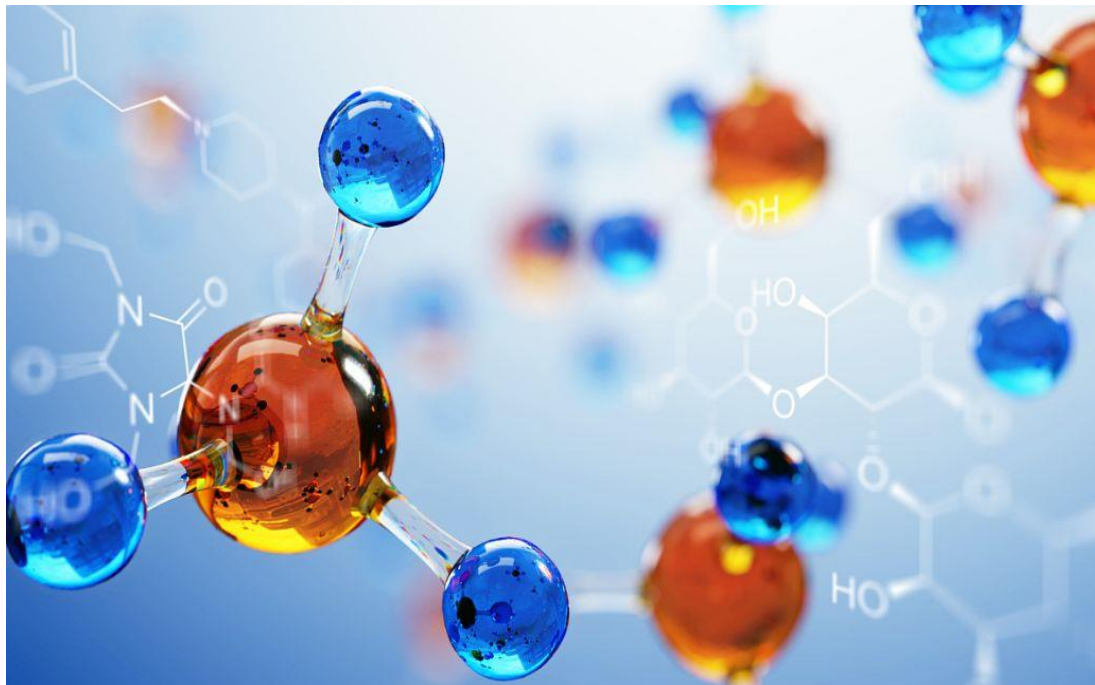
(4) C标准函数应用

用户程序需要使用标准函数时，只需要使用前用 #include 包含该标准函数所需的系统头文件即可。

1.3

C语言的基本语法单位

- 1) 标识符
- 2) 关键字
- 3) 分隔符与注释符



- 标识符:在高级语言程序中由用户(即程序员)或编译程序(有时称系统)定义的常量、变量、自定义的数据类型、函数等的**名字**。
- 标识符的组成规则 :**由字母(A~Z, a~z)、下划线_和数字(0~9)组成, 其第一个字符必须为字母或下划线_**。字母区分大小写。
- 标识符的有效长度: 能够被编译程序识别的标识符的字符的数目称为标识符的有效长度。

1.3.2

关键字

- 关键字由固定的小写字母组成，是系统预定的名字，用于表示C语言的语句，数据类型、存储类型或运算符。关键字又称为保留字。
- 标准C定义的32个关键字如下：

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

分隔符: 包括空格符、制表符、换行符、换页符。程序中两个相邻的标识符、关键字和常量之间必须用分隔符分开(通常用空格符)。

注释符:

1. `/* ..... */`

2. `//`

一个完整的计算机程序,常常要求具备输入输出功能。

C语言程序的输入输出功能是通过调用系统提供的标准函数实现的。

几个常用的C语言输入输出函数:

printf函数

scanf函数

getchar函数

putchar函数

1.4.1

格式化输出printf函数

一般使用形式如下:

`printf(“输出格式”,输出项系列);`

常用输出格式转换说明符:

<code>%d</code>	十进制整数
<code>%x</code>	十六进制整数
<code>%f</code>	浮点数(实数)
<code>%c</code>	单一字符
<code>%s</code>	字符串

如: `printf(“v=%f\n”, v);`

把输出项v的值按%f规定的浮点小数形式输出。

如: `printf(“h=%d\n”, h);`

把输出项h的值按%d规定的十进制整数形式输出。

1.4.1

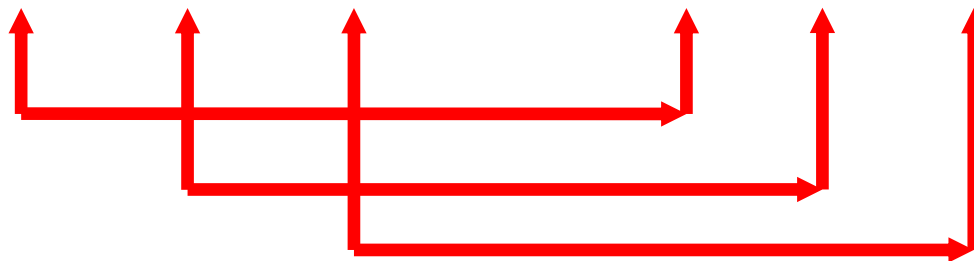
printf函数多项输出使用例



要求输出格式中的转换说明符与输出项的个数必须相同。它们按各自的先后顺序——对应。

如：

```
printf(".... %d... %x... %f...", a, b, c);
```



1.4.1

printf函数

- 输出格式中除转换说明符以外的其它字符都原封不动地输出到标准输出设备。
- 其中以' \ ' 打头后跟一个字母或数字的部分称为换码序列(转义字符)。

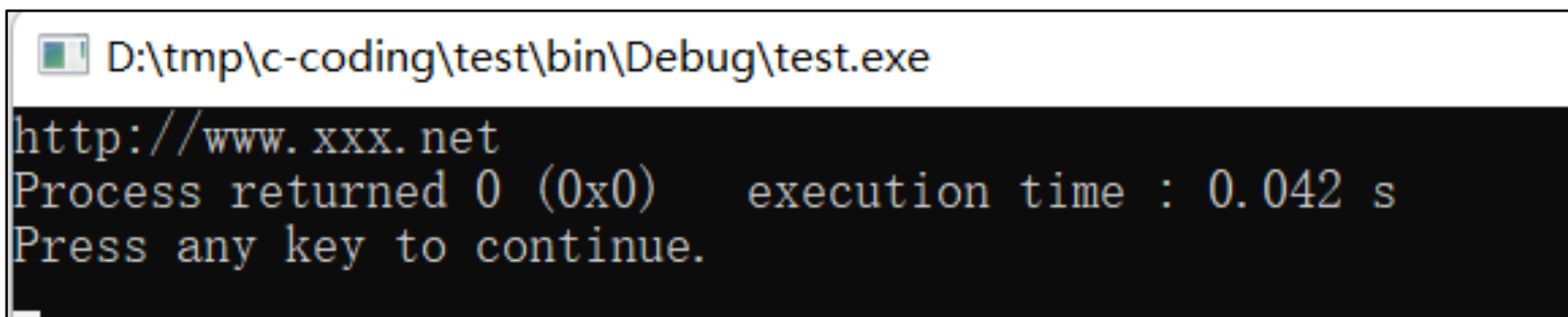
字符类型	字符表示	字符含义	ASCII码值	"\ddd"表示	"\xhh"表示
部分转义字符	'\n'	回车换行	10	\012	\x0A
	'\t'	制表符	9	\011	\x09
	'\f'	走纸换页	12	\014	\x0C
	'\0'	空字符	0	\000	\x00
	'\b'	退格	8	\010	\x08
	'\r'	回车	13	\015	\x0D

1.4.1

printf函数

- 输出格式中除转换说明符以外的其它字符都原封不动地输出到标准输出设备。
- 其中以' \ ' 打头后跟一个字母或数字的部分称为换码序列(转义字符)。

```
1 #include <stdio.h>
2 int main( )
3 {
4     printf("\x68\x164\x164\x70: //www. xxx. \x6e\x145\x74");
5     return 0;
6 }
```



```
D:\tmp\c-coding\test\bin\Debug\test.exe
http://www.xxx.net
Process returned 0 (0x0)    execution time : 0.042 s
Press any key to continue.
```

1.4.2

格式化输入scanf函数

- 其一般使用形式如下：

`scanf("输入格式", 输入项系列);`

- 输入格式中一般只使用转换说明符，否则容易出错。
- 输入项必须是地址量，(变量名前加上&表示变量的地址)。

1)常规输入例:

`scanf("%d",&a);`

输入10

结果: a=10

2)输入中有分隔符例:

`scanf("%d:%d",&a,&b);`

输入3:5

结果: a=3,b=5

3)输入长度给定例:

`scanf("%4d%2d%2d",&a,&b,&c);`

假设输入序列为: 20160125

结果: a=2016,b=01,c=25



输入数据时，遇到下列情况时该数据认为结束

a.遇空格、回车或者tab键

b.遇宽度结束，如“%3d”只取输入项前三列。

1.4.3

字符输入getchar函数

getchar函数的功能是从键盘读入一个字符。在程序中必须用一个对应的变量接收读取的字符，如下所示：

```
c = getchar();
```

```
1  #include <stdio.h>
2  void main()
3  {
4      char c;
5      printf("请输入字符: ");
6      c = getchar();
7
8      printf("输入的字符: ");
9      putchar(c);
10     printf("\n%d, %c", c, c);
11 }
12
```

```
请输入字符: a
输入的字符: a
97, a
```

```
请输入字符: A
输入的字符: A
65, A
```

`putchar`函数的功能是把一字节的代码值所代表的字符输出到标准输出设备上，它的应用形式如下：

`putchar(c);`

它把变量c的值作为代码值，把该代码值对应的字符输出到标准输出设备。

1.4.5

getchar函数的怪现象

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char ch1, ch2;
6      ch1 = getchar();
7      printf("ch1=%c\n", ch1);
8      ch2 = getchar();
9      printf("ch2=%c\n", ch2);
10     return 0;
11 }
12
```

D:\test-c\studyDebug\bin\Debug\studyDebug.exe

```
q
ch1=q
ch2=
```

```
Process returned 0 (0x0)   execution time : 7.123 s
Press any key to continue.
```



输入缓冲队列

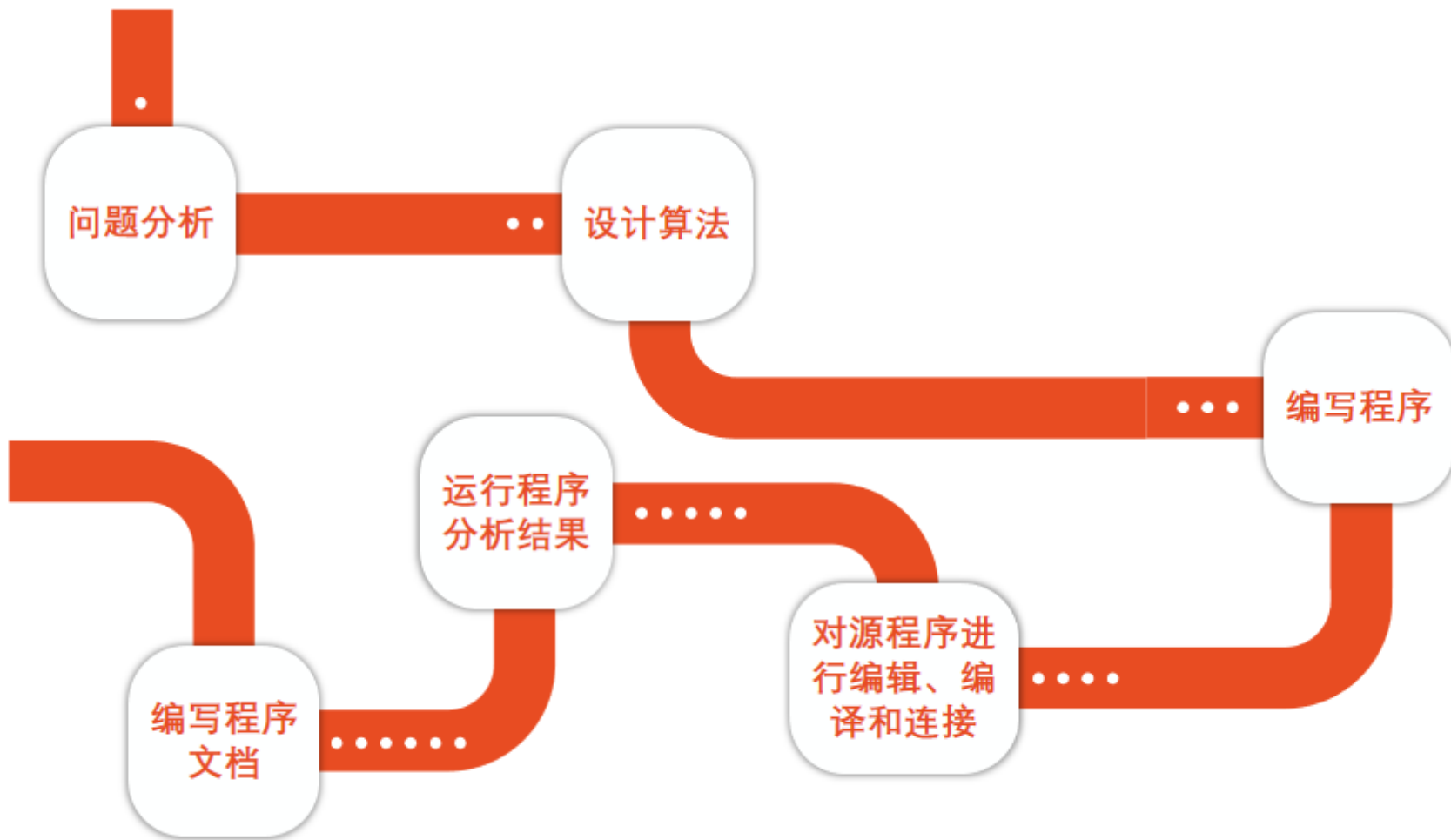
\n b a



(b) Queue waiting for a bus.

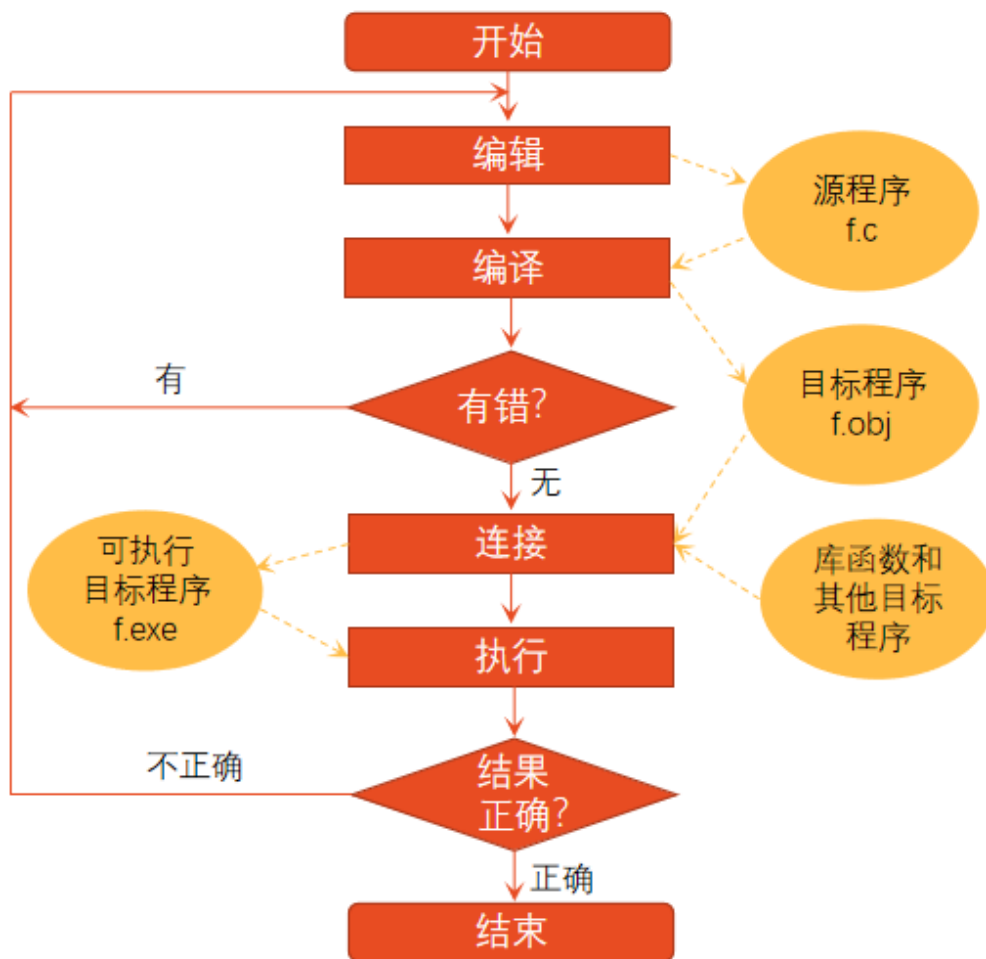
1.5

程序设计的任务



1.5

运行C程序的一般步骤

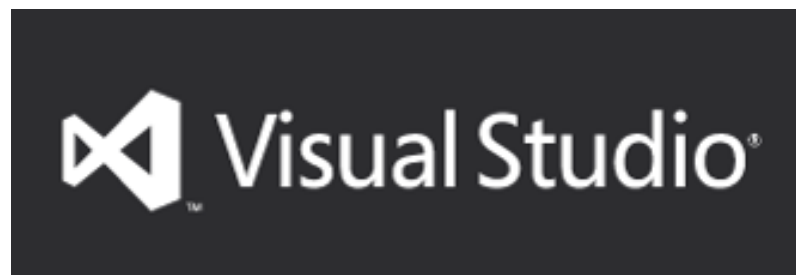
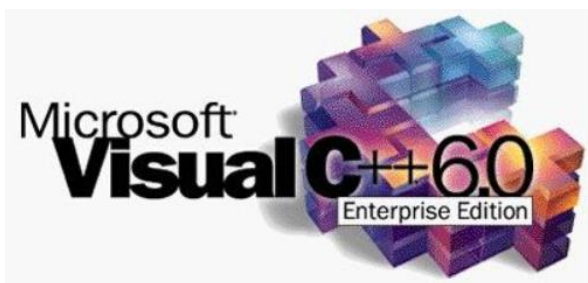


1.5

集成开发环境IDE

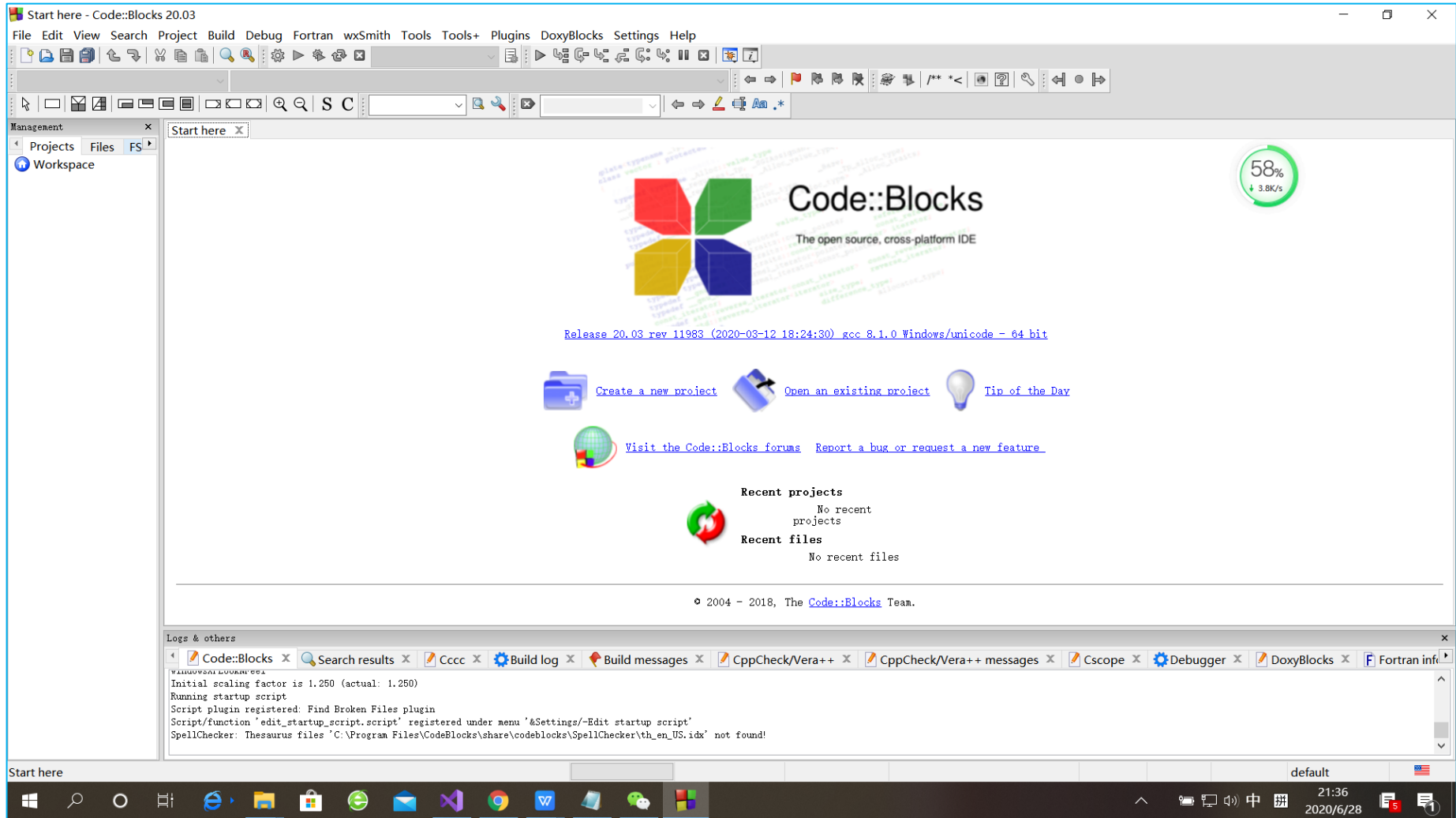
- (1) 编辑器
- (2) 编译器

本课程仅使用Code::Blocks



1.5

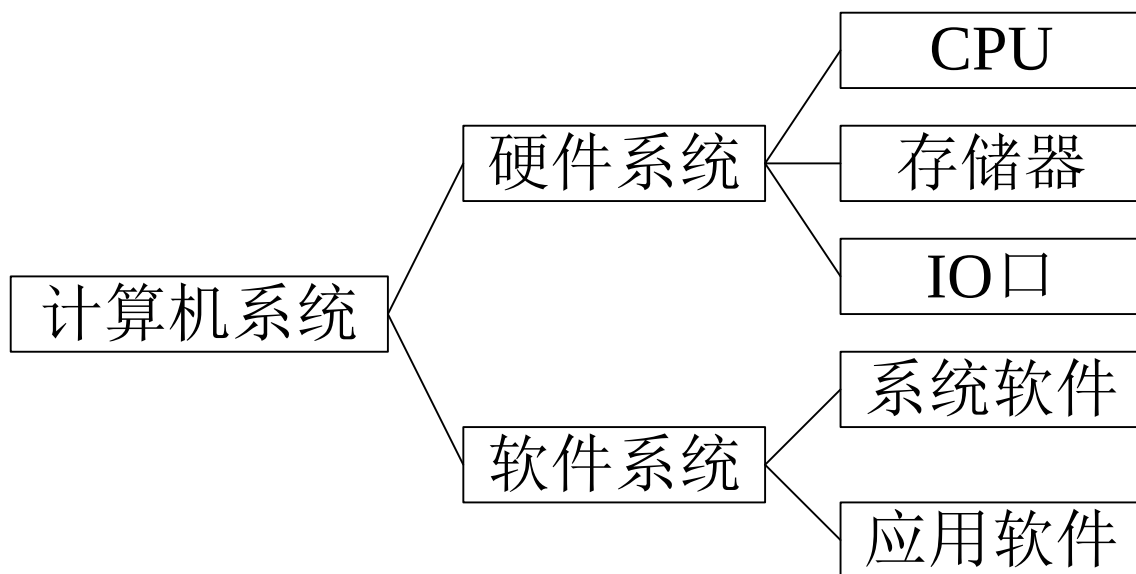
Code::Blocks



本章小结

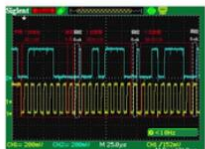
- 掌握C语言入门的基础知识;
- 了解C语言的发展历程及特点;
- 熟悉和掌握C语言程序的书写格式、结构特点和组成
C语言的基本语法单位;
- 掌握几种经常使用的输入输出标准函数;
- 了解运行C程序的一般步骤。

计算机系统包含CPU(central processing unit)、存储器和输入/输出接口等电路模块构成硬件部分，还有软件系统。





十进制



二进制

$$(a_n a_{n-1} \dots a_1 a_0 . c_1 c_2 c_3 \dots)_b = \sum_{k=0}^n a_k b^k + \sum_{k=1}^{\infty} c_k b^{-k}$$

$$\begin{aligned} & (a_{n-1} a_{n-2} \dots a_1 a_0 . a_{-1} \dots a_{-m})_2 \\ &= a_{n-1} \times 2^{n-1} + a_{n-2} \times 2^{n-2} + \dots + a_1 \times 2^1 + a_0 \times 2^0 + a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \dots + a_{-m} \times 2^{-m} \end{aligned}$$

【例】：将二进制数据111.01写成加权系数的形式。

$$\text{解: } (111.01)_2 = (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) + (0 \times 2^{-1}) + (1 \times 2^{-2})$$

- 基数为16的数制为十六进制，有0-9、A、B、C、D、E、F十六个数字字符(其中A-F分别表示10—15)，逢十六进一。
- 一个十六进制数N可以表示为：

$$\begin{aligned}(N)_{16} &= K_{n-1} \times 16^{n-1} + K_{n-2} \times 16^{n-2} + \cdots + K_1 \times 16^1 + K_0 \times 16^0 \\ &\quad + K_{-1} \times 16^{-1} + \cdots + K_{-m} \times 16^{-m} \\ &= \sum_{i=-m}^{n-1} K_i \times 16^i\end{aligned}$$

- 二进制数的位数多，书写和阅读很不方便，容易出错。微机中通常采用十六进制数作为二进制数的缩写形式，书写简单，一目了然。**4位二进制对应1位十六进制**

0x2f, 0xef

计算机中二进制数的存储单位

1、位 (Bit)

计算机所能表示的最小数据单位，即1位二进制数1或者0。

2、字节 (Byte)

8位二进制数称为一个字节。最基本的数据单位。

3、字 (Word)

16位二进制数称为一个字。1个字有2个字节。

机器数和真值

在计算机中，用“0”表示“+”号，用“1”表示“-”号。这样的数称为**机器数**，而把机器数所代表的数的实际值称为机器数的**真值**

例如，真值 $x_1 = +1001001$ $x_2 = -1001001$

机器数 $x_1 = 01001001$ $x_2 = 11001001$

无符号数与有符号数的机器数表示

无符号数只表示数值大小，常用于表示无正负概念的数、代码和存储器的地址

有符号数具有正负的概念，用“0”或者“1”放到表示数值大小的数码序列最前面来表示正负数

计算机中带符号数有三种表示方法：原码、反码、补码

在这三种机器数的表示形式中，符号位的规定相同，仅是数值部分的表示形式不同

原码

正数的符号位用“0”表示，负数的符号位用“1”表示，其余数字位表示数值本身

例， $x_1 = +1101101$ $[x_1]_{\text{原}} = 01101101$

$x_2 = -1101101$ $[x_2]_{\text{原}} = 11101101$

- 0在原码表示中有下列两种形式：

$[+0]_{\text{原}} = 0000\ 0000$ $[-0]_{\text{原}} = 10000000$

- 数0的原码表示不是唯一的。对于8位二进制数来说，原码可表示的数的范围是-127~+127

反码

- 正数的反码和正数的原码相同
- 负数的反码其符号位为1，其余各位逐位取反，即0变为1，1变为0

例如，

$$x_1 = +1101101 \quad [x_1]_{\text{原}} = 01101101$$

$$[x_1]_{\text{反}} = 01101101$$

$$x_2 = -1101101 \quad [x_2]_{\text{原}} = 11101101$$

$$[x_2]_{\text{反}} = 10010010$$

补码

- 正数的补码与正数的原码相同
- 负数的补码等于负数的反码加1

例如：

$$x_1 = +1101101 \quad [x_1]_{\text{原}} = 01101101$$

$$[x_1]_{\text{反}} = 01101101$$

$$[x_1]_{\text{补}} = 01101101$$

$$x_2 = -1101101 \quad [x_2]_{\text{原}} = 11101101$$

$$[x_2]_{\text{反}} = 10010010$$

$$[x_2]_{\text{补}} = 10010011$$

例如：求-3的补码

符号
位不
变

原码

按位取反

加一后得到补码

例如：求-15的补码

符号
位不
变

原码

按位取反

加一后得到补码

例如：求-3的补码

符号 位不 变	原码	1 0 0 0	0 0 1 1
	按位取反	1 1 1 1	1 1 0 0
	加一后得到补码	1 1 1 1	1 1 0 1

例如：求-15的补码

符号 位不 变	原码	1 0 0 0	1 1 1 1
	按位取反	1 1 1 1	0 0 0 0
	加一后得到补码	1 1 1 1	0 0 0 1

计算机如何执行下面运算：

$$-3-15$$

$$1-1$$

$$-1-127$$

$$127+1$$

$$127+2$$

$$-1-128$$